

IS4103

Operating Systems

Memory Management

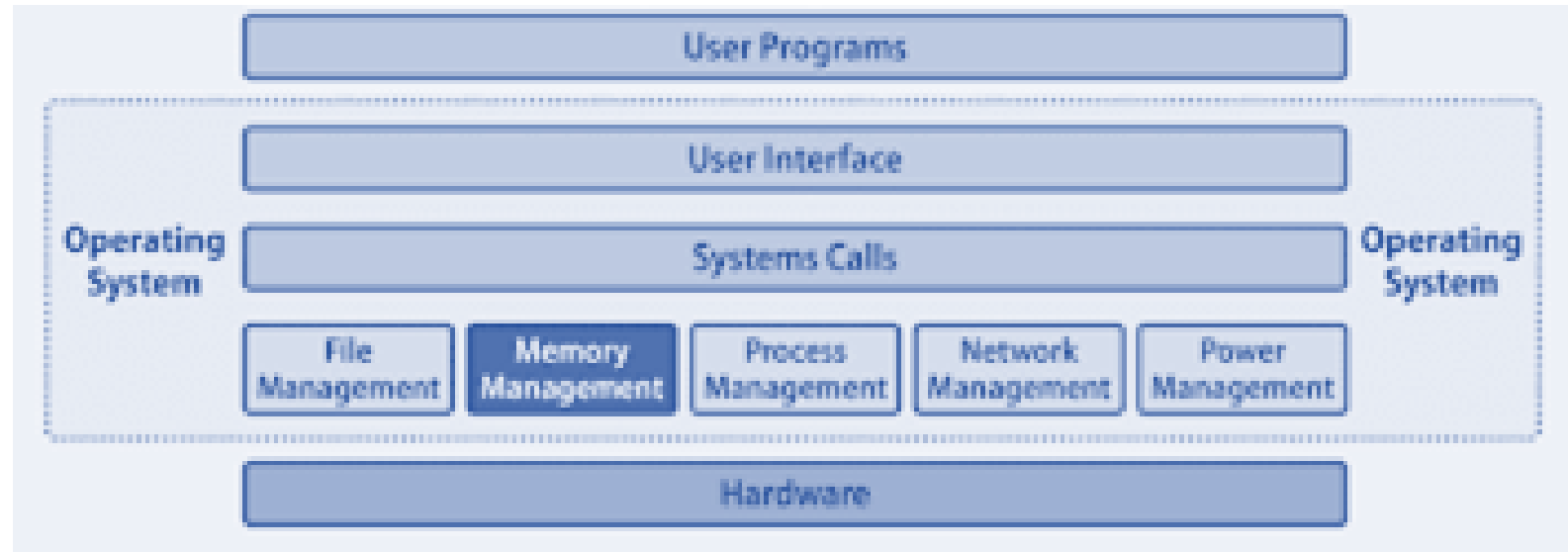
T.Kayalvizhy

Learning Objectives

Learning Outcome	Description
ILO3	Evaluate the performance of various scheduling algorithms

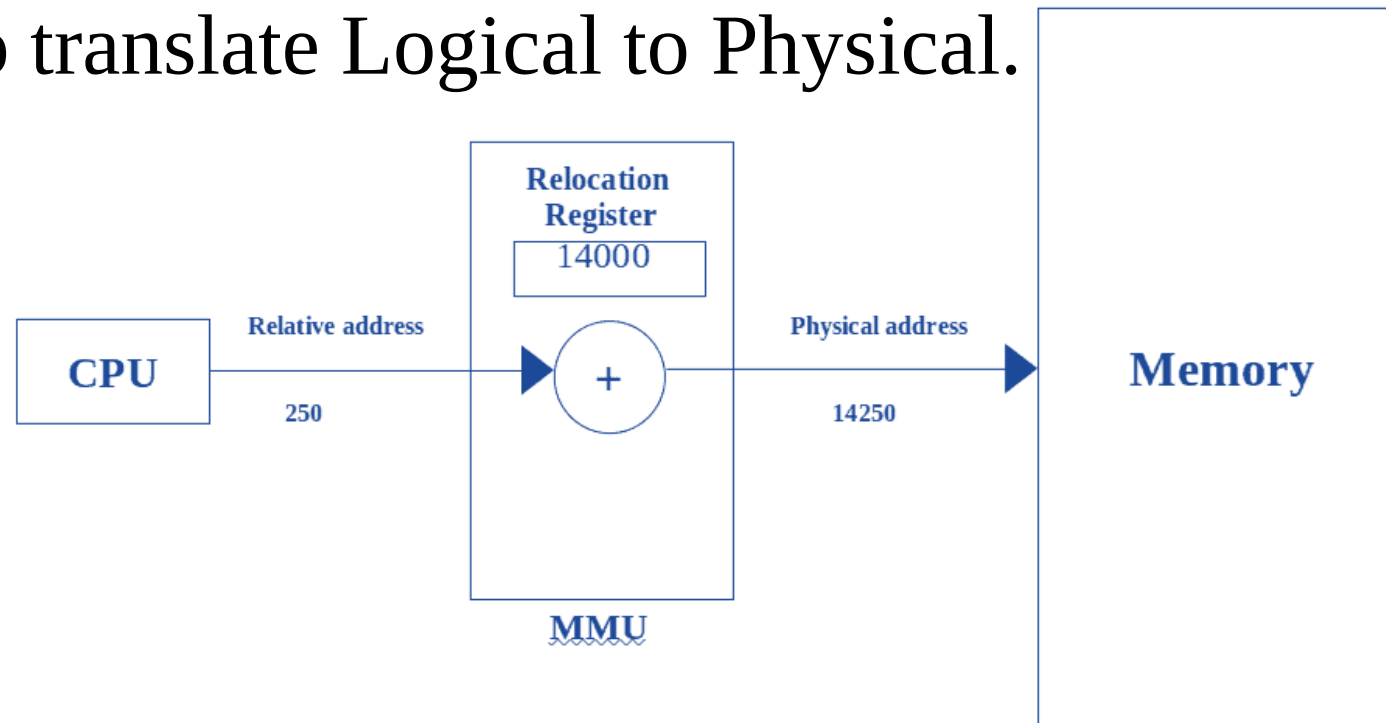
Memory Management

- Memory management is the OS function responsible for:
 - Allocating memory to programs
 - Deallocating memory after use
 - Tracking memory usage



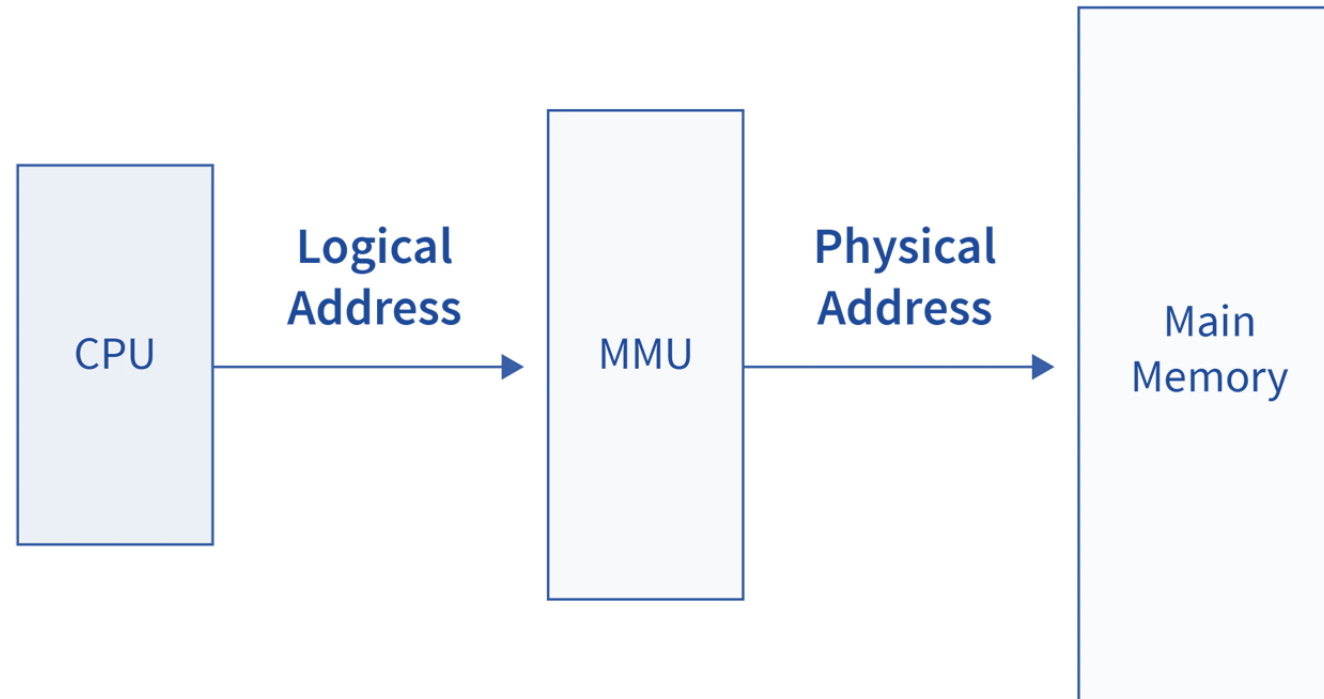
Logical vs. Physical Address Space

- Logical Address: Generated by the CPU (also called "Virtual Address"). The programmer sees this.
- Physical Address: The actual location in the memory unit (RAM).
- The Bridge: The Memory Management Unit (MMU) sits between the CPU and RAM to translate Logical to Physical.



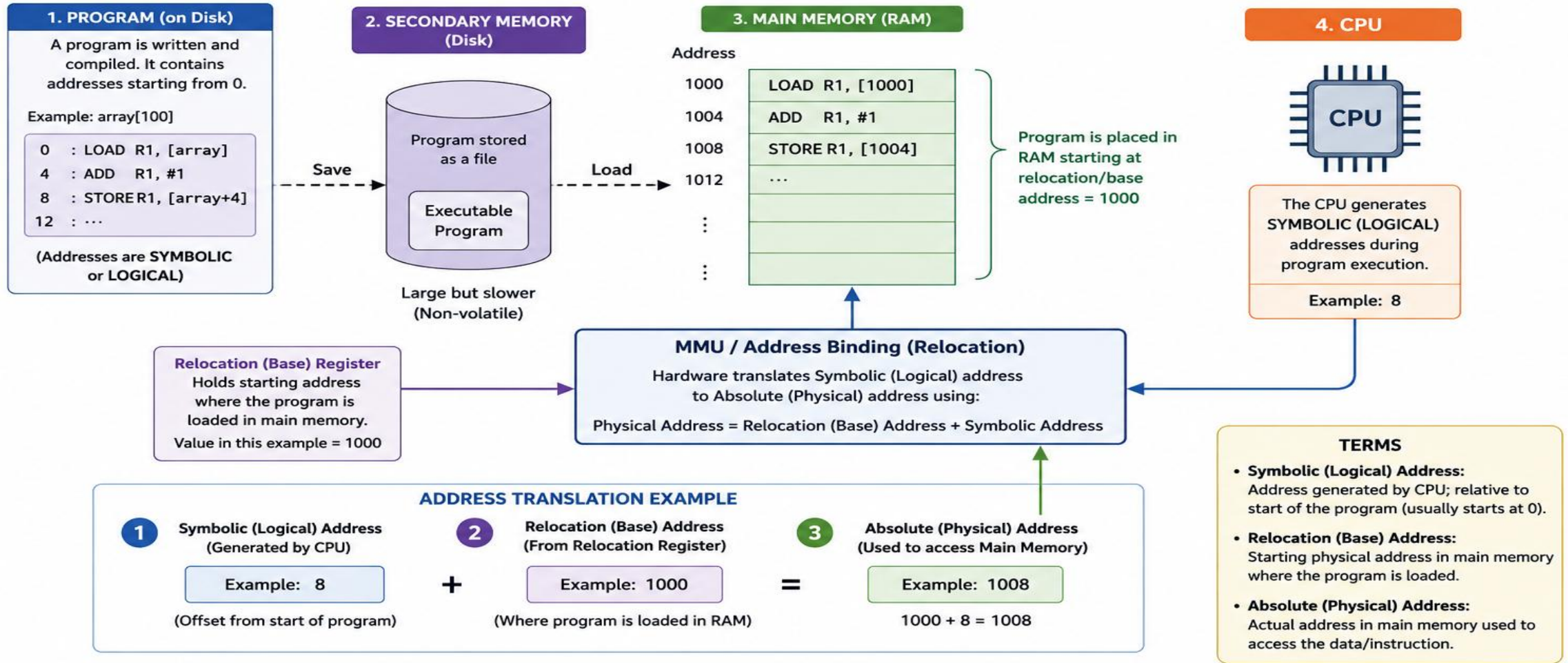
Address binding

- Address binding is the process of mapping program instructions and data to actual locations in computer memory.



Address binding

Program (on Disk) → Load to Main Memory → Bind Addresses → Execute on CPU

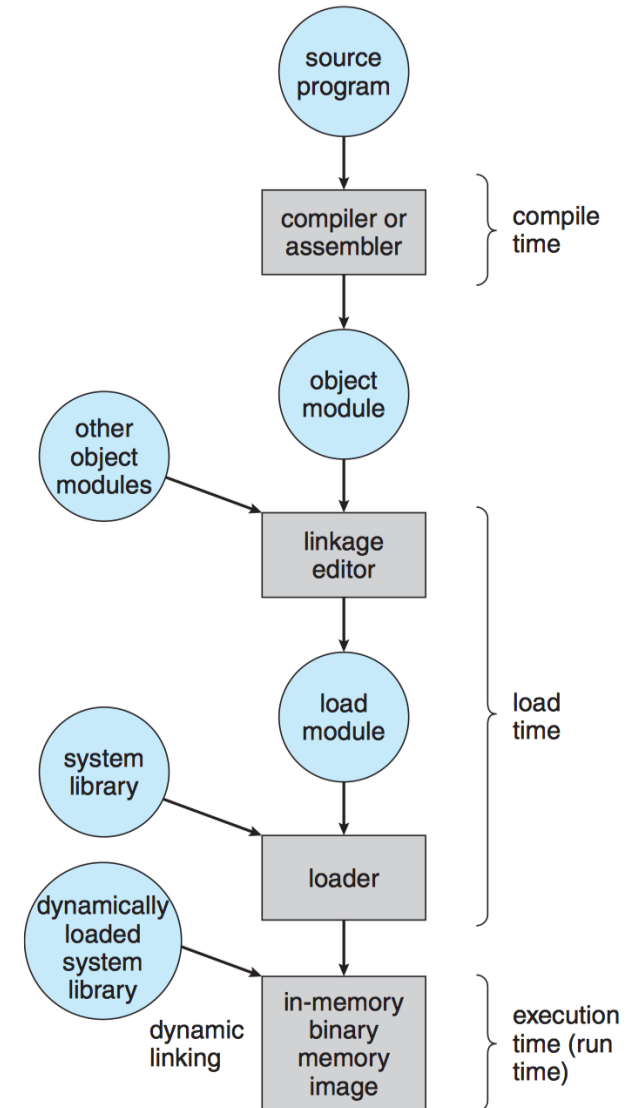


SUMMARY

1. Program is stored in Secondary Memory (Disk).
2. When loaded into Main Memory, it is placed at some Base (Relocation) Address generated by CPU.
3. During execution, CPU generates Symbolic addresses.
4. MMU adds the Base address to Symbolic address to produce Absolute address.
5. Main Memory is accessed using Absolute address.

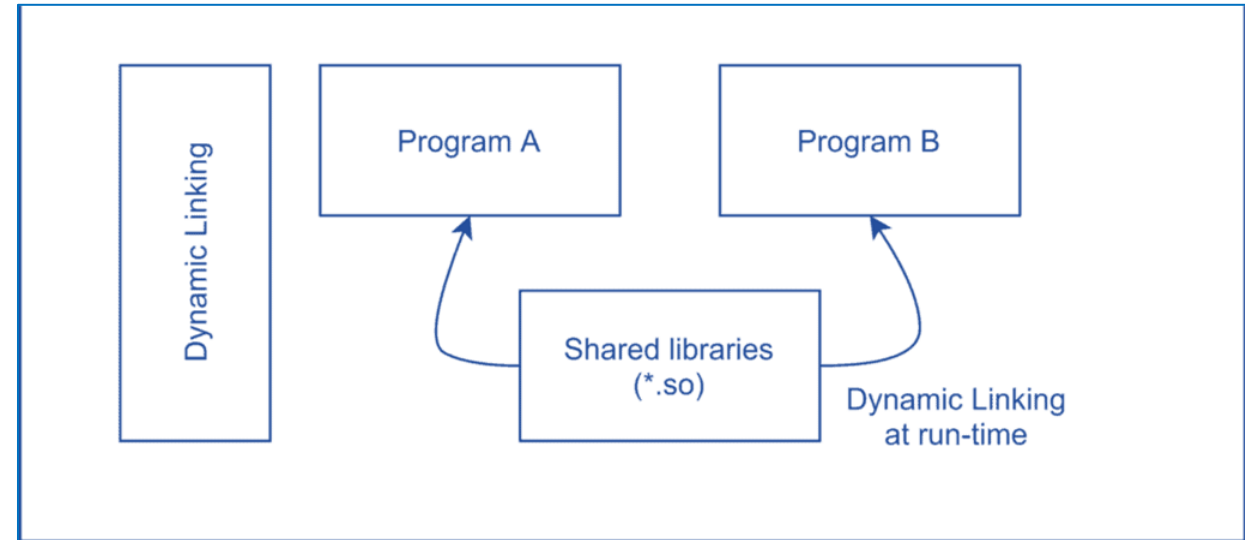
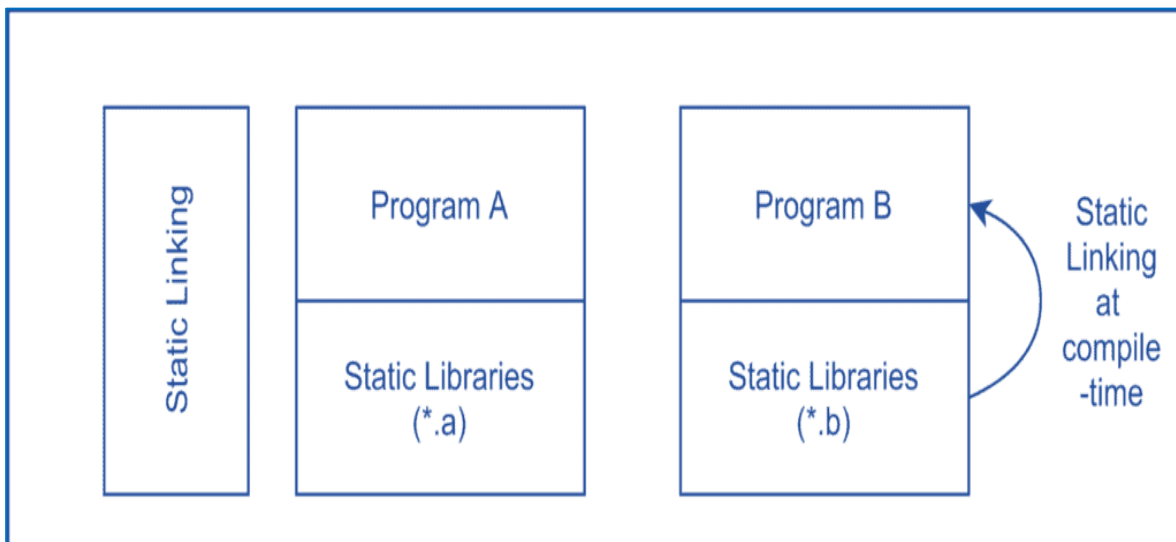
Types of Address Binding (Stages)

- **Compile-Time Binding:** The compiler translates symbolic addresses to absolute addresses (e.g., location 1000).
- **Load-Time Binding:** The compiler generates relocatable code. The final physical address is decided only when the program is loaded into memory.
- **Execution-Time (Run-Time) Binding:** The process can be moved during its execution from one memory segment to another. This requires hardware support (like MMU).



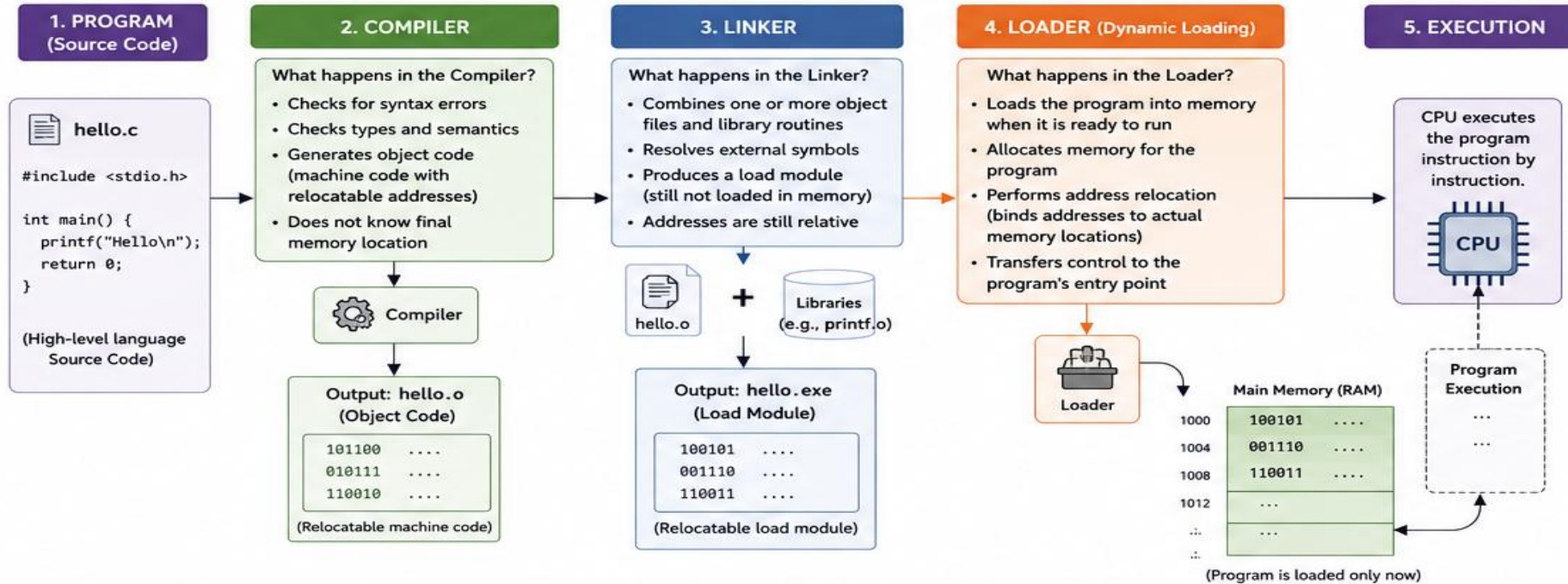
Dynamic Loading & Linking

- **Dynamic Loading:** A routine is not loaded until it is called. This saves space.
- **Dynamic Linking:** Rather than including library code in the executable, a "stub" is included. The actual linking happens at runtime (e.g., .DLL files in Windows).



DYNAMIC LOADING: OVERVIEW

A program is loaded into memory only when it is ready to execute.



SAMPLE PROGRAM (hello.c)

```
1 #include <stdio.h>
2
3 int add(int a, int b) {
4     return a + b;
5 }
6
7 int main() {
8     int x = 10, y = 20;
9     printf("Sum = %d\n", add(x, y));
10    return 0;
11 }
```

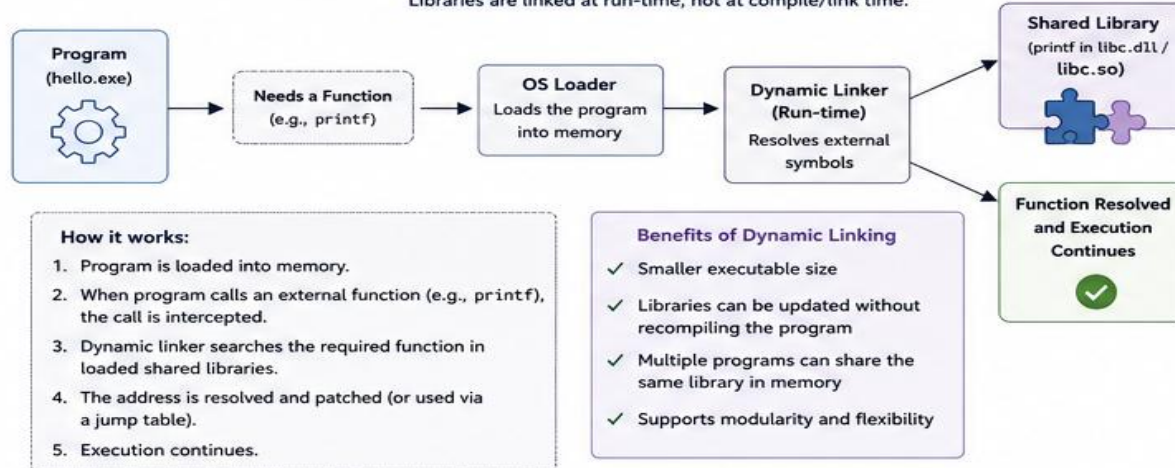
WHAT HAPPENS IN THE COMPILER (DETAIL)

1. Lexical Analysis → breaks code into tokens
2. Syntax Analysis → checks grammar (parsing)
3. Semantic Analysis → checks meaning (types, declarations, etc.)
4. Intermediate Code Generation → creates intermediate representation
5. Code Optimization → improves efficiency
6. Code Generation → produces object code (relocatable machine code)

Output: hello.o (object code with relocatable addresses and symbol table)

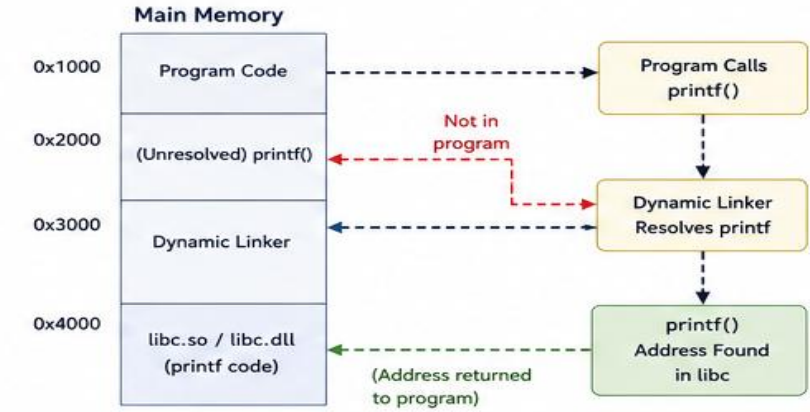
DYNAMIC LINKING

Libraries are linked at run-time, not at compile/link time.



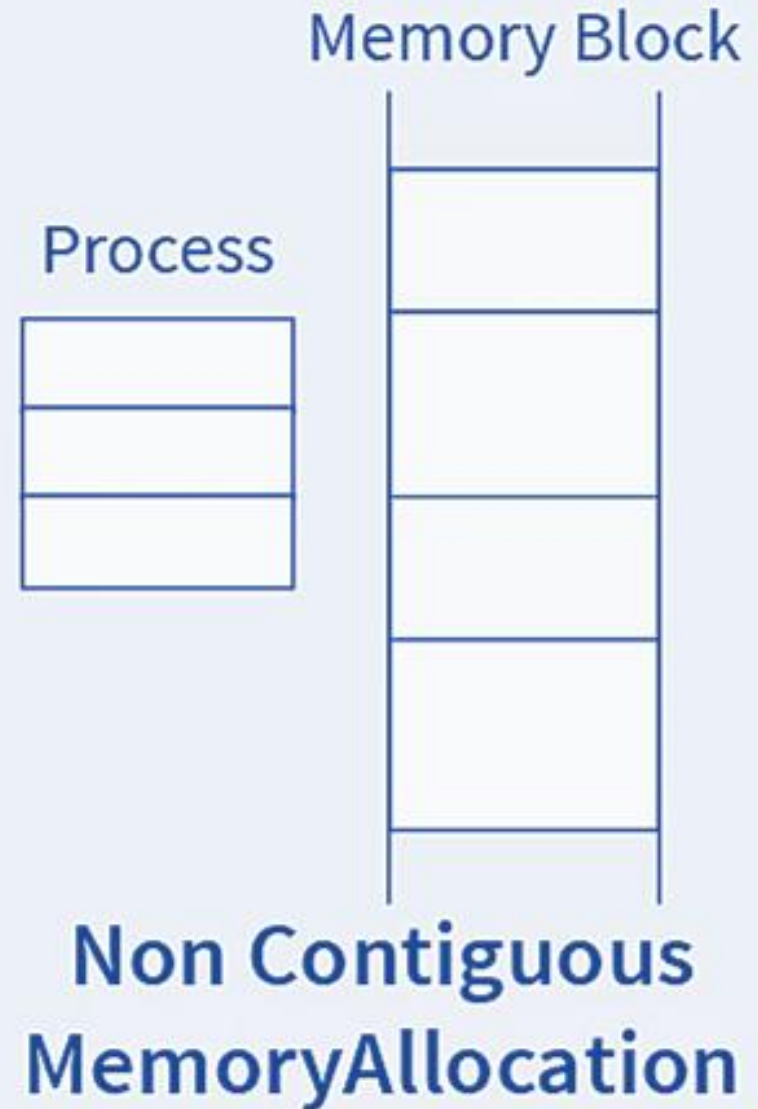
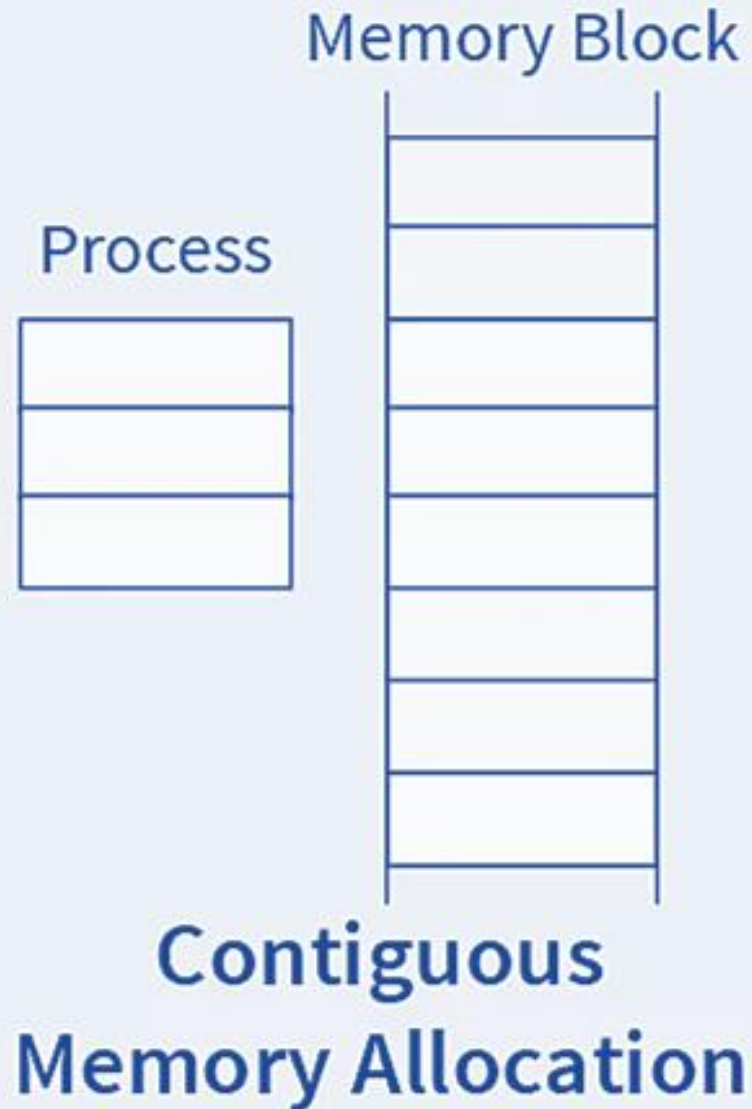
EXAMPLE FLOW (Standard Library)

1. Program calls printf("Hello")
2. printf is not in the program → control goes to dynamic linker
3. Dynamic linker looks in loaded libraries (libc.so / libc.dll)
4. Finds printf address
5. Returns control to program
6. printf executes
7. Control returns to program
8. Program continues

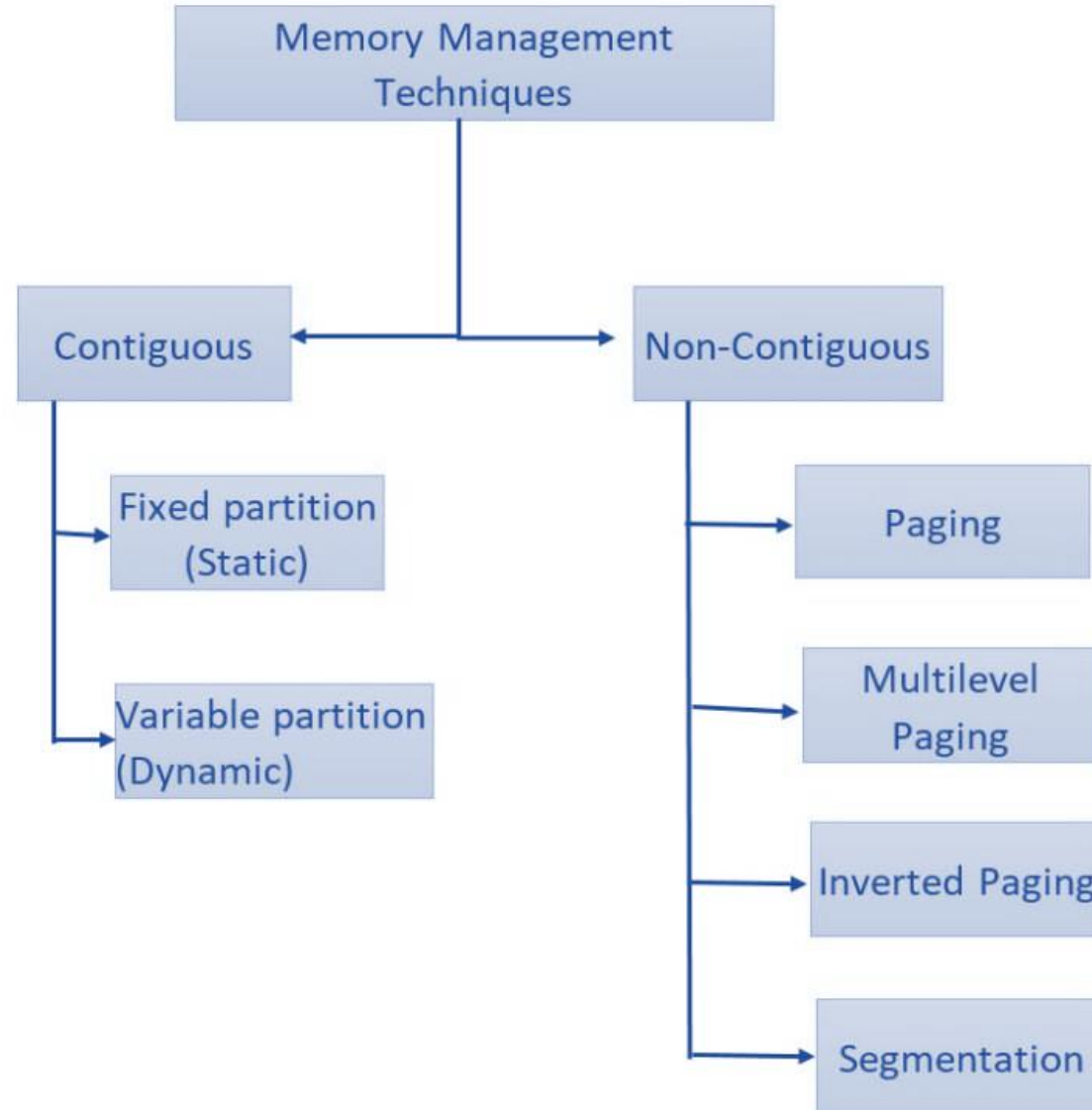


Note: In some systems, the first call to a function goes through the dynamic linker. Later calls may go directly to the resolved address (Procedure Linkage Table - PLT).

Contiguous & Non-Contiguous Memory Allocation



Contiguous & Non-Contiguous Memory Allocation

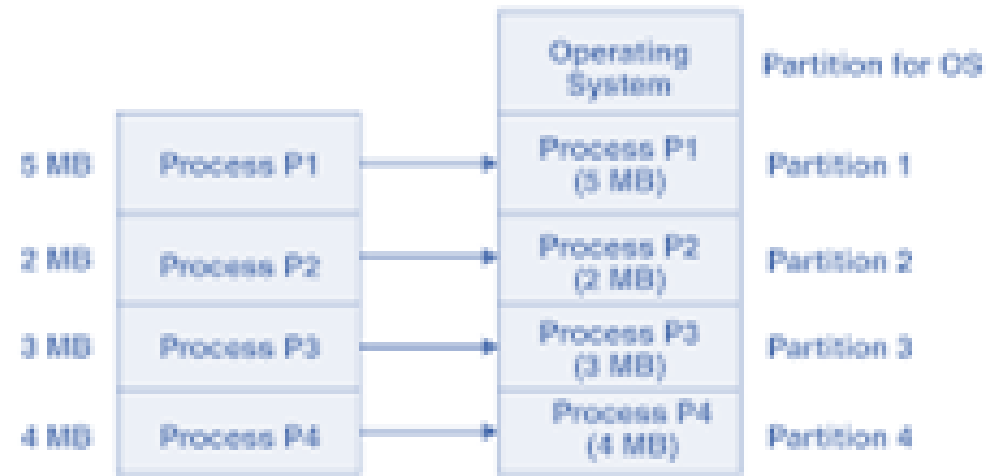
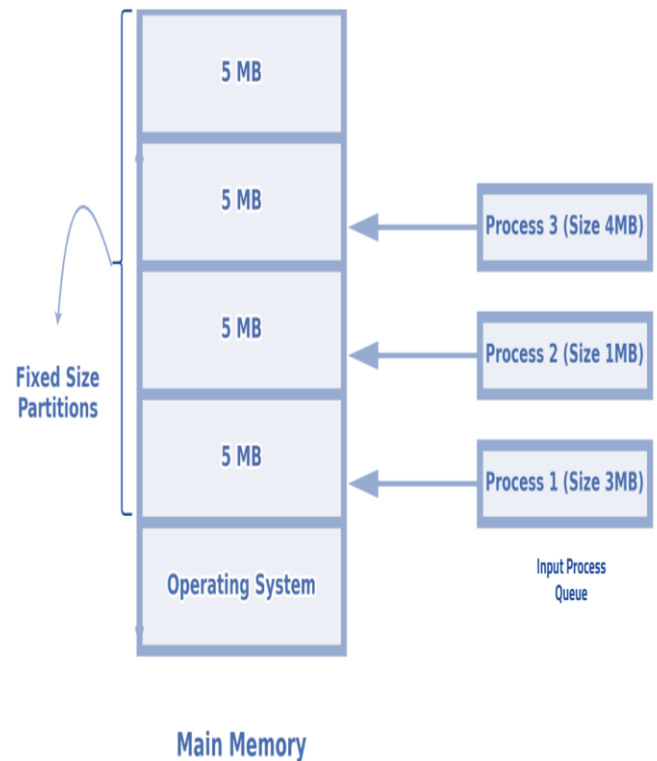


Contiguous & Non-Contiguous Memory Allocation

Feature	Contiguous Memory Allocation	Non-Contiguous Memory Allocation
Storage Style	The process is stored in one single, continuous block of RAM.	The process is divided into parts (Pages/Segments) and scattered in RAM.
Hardware	Requires simple Base and Limit registers .	Requires a Page Table or Segment Table and specialized hardware (MMU).
Fragmentation	Suffers heavily from External Fragmentation .	Eliminates External Fragmentation; may have tiny Internal Fragmentation .
OS Overhead	Low. The OS only tracks the start address and the size.	High. The OS must maintain a mapping table for every single process.
Execution	Faster address translation (simple addition).	Slightly slower due to table lookups (minimized by TLB hardware).
Swapping	Difficult, as the OS must find a single large hole to bring a process back.	Easy, as the OS can bring pieces back into any available small frames.

Contiguous Memory Allocation

- Contiguous memory allocation is an OS technique where each process is assigned a single, unbroken block of main memory.



Advantages and Disadvantages of Equal Size Partitioning

Advantages	Disadvantages
Simple implementation	Internal fragmentation
Easy allocation	Large processes cannot fit
Easy memory tracking	Poor memory utilization
Fast allocation process	External fragmentation

Equal Size Partitioning

Question

A system has:

Total memory = 1000 KB

5 equal partitions

Processes:

P1 = 120 KB

P2 = 170 KB

P3 = 220 KB

P4 = 80 KB

Find partition size, allocate processes, and calculate internal fragmentation

Advantages and Disadvantages of Variable-Size Fixed Partitioning

Advantages	Disadvantages
Better memory utilization	Small partitions may remain unused
Supports different process sizes	Still causes internal fragmentation
Reduced internal fragmentation	Complex partition selection
More flexible than equal partitioning	Difficult memory management

Fixed Partitioning

Question

Memory Partitions:

100 KB

200 KB

300 KB

500 KB

Processes:

P1 = 90 KB, P2 = 260 KB, P3 = 180 KB, P4 = 470 KB

Find partition size, unallocated processes, and calculate internal fragmentation

Overlay Technique

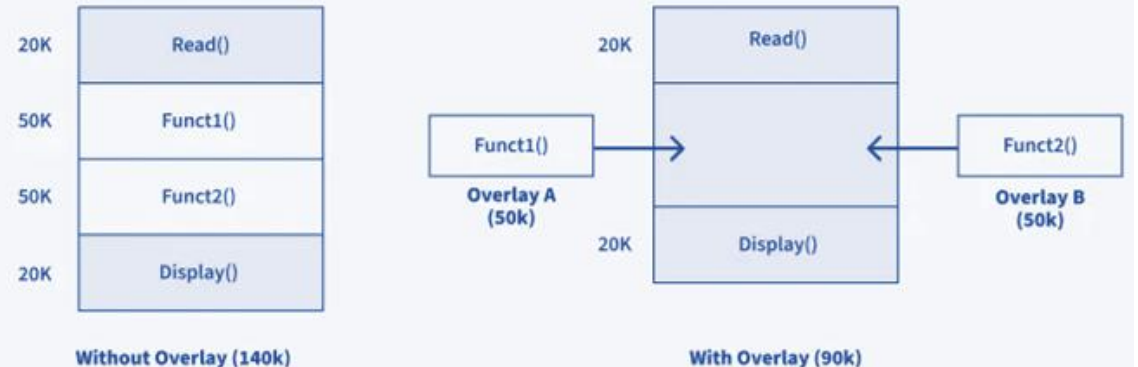
Overlay allows programs larger than memory to execute by loading only the required parts.

Concept

Only the required module is loaded into memory

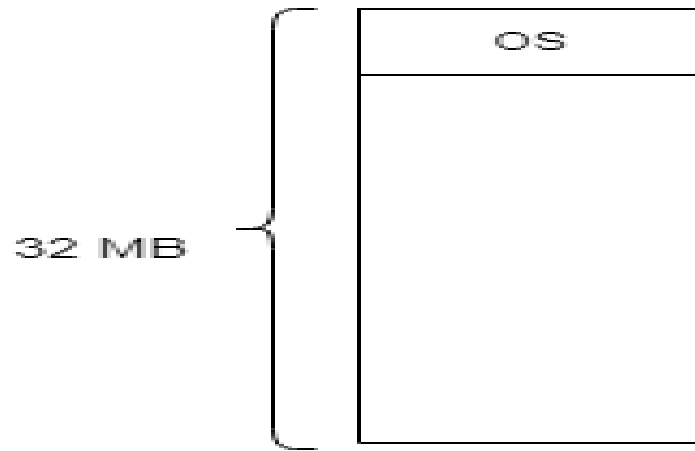
Unused module removed

New module loaded

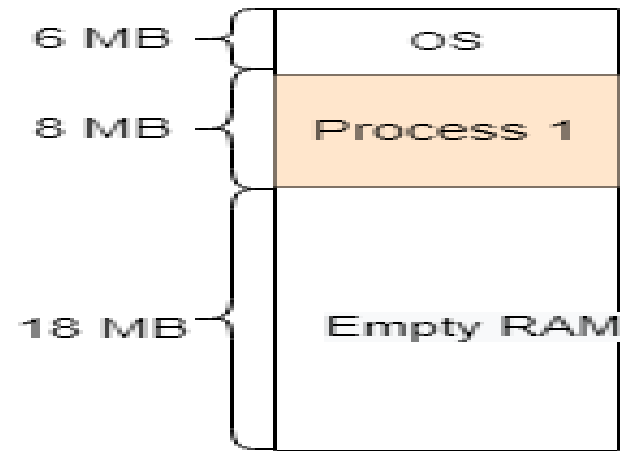


Dynamic Partitioning

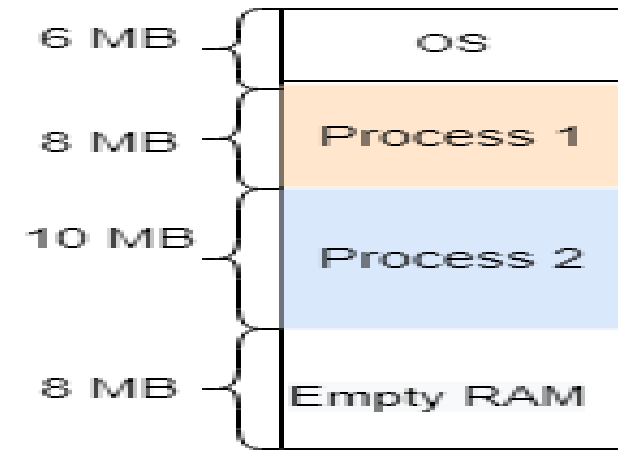
RAM



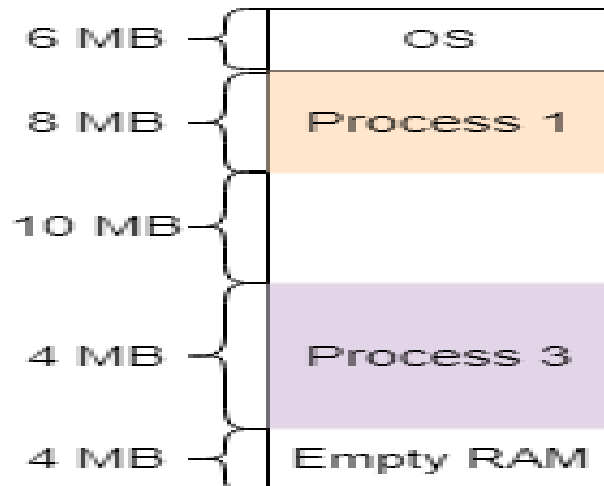
Fig(i)



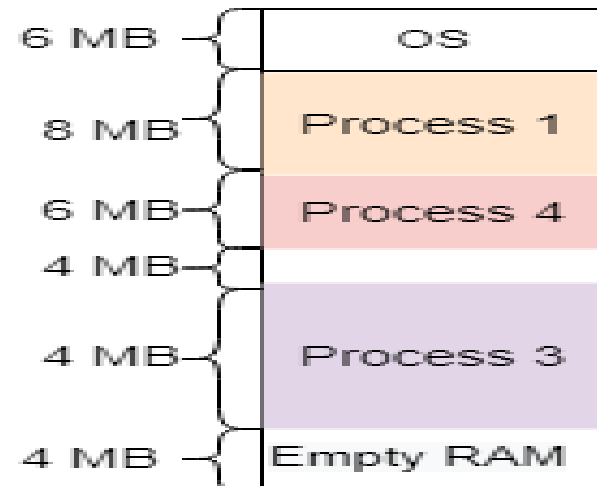
Fig(ii)



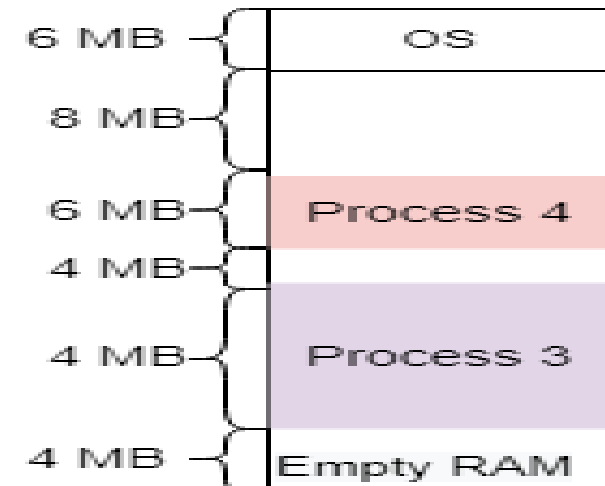
Fig(iii)



Fig(iii)



Fig(iv)



Fig(v)

Dynamic Partitioning

Partitions are created dynamically according to process size.

- Memory allocated when needed
- Partition size changes dynamically
- Better memory utilization

Advantages	Disadvantages
No internal fragmentation	External fragmentation occurs
Efficient memory utilization	Compaction may be required
Flexible allocation	Allocation complexity increases
Better support for varying process sizes	Slower allocation process

Tracking Dynamic Contiguous Allocation

Bitmaps: Fixed Tracking

Fixed Granularity: Memory is divided into small units; one bit tracks if a unit is free (0) or used (1).

Compactness: Requires very little memory overhead (1 bit per unit) compared to full data structures

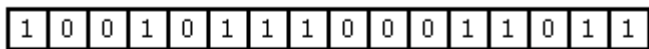
Search Efficiency: Finding the first free unit is fast via bit-level scanning, though finding specific lengths can be $O(n)$

Linked Lists: Logical Segments

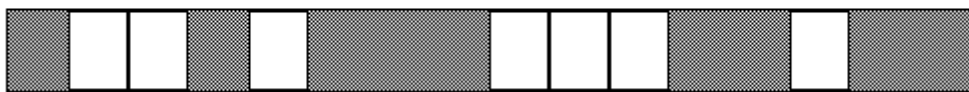
Segment Logic: Tracks "Process" and "Hole" segments as nodes with [Type, Start Address, Length].

Efficient Coalescing: When a process terminates, merging adjacent holes is trivial by updating list pointers.

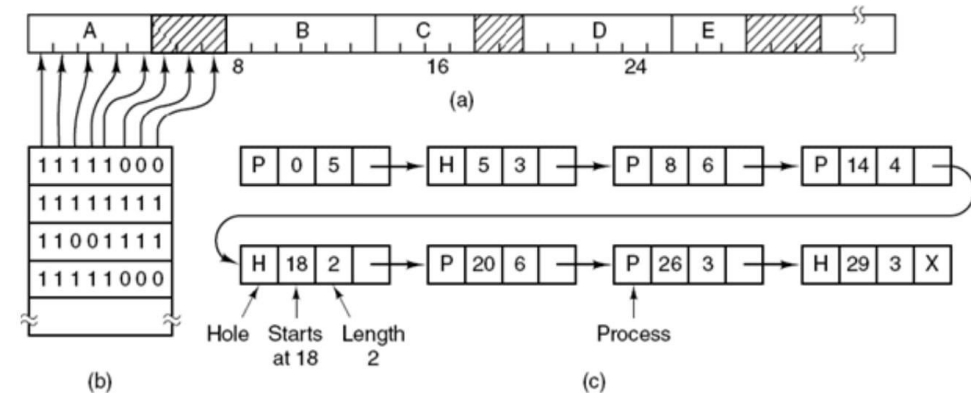
Bit Map:



Memory:



16 bits handles 16K of memory with the chunk (page) size of 1K



Eliminating External Fragmentation

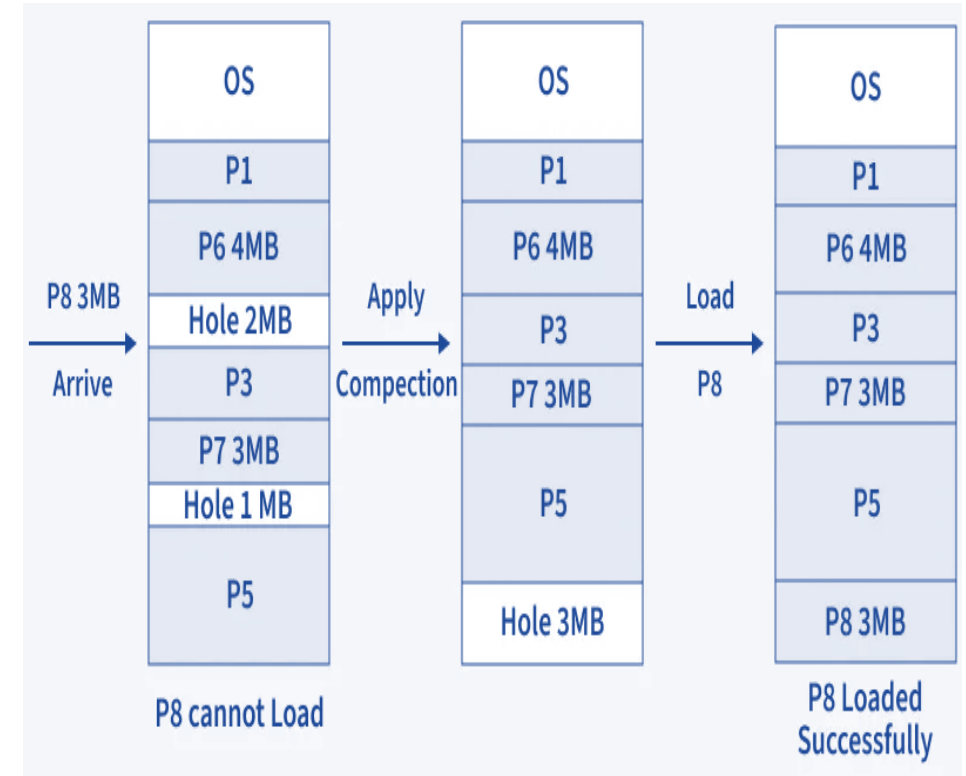
Memory Shuffling: The OS moves all allocated processes to one end of memory, merging all free spaces into one large contiguous block.

Execution Requirement: Can only be performed if relocation is dynamic and done at execution time via relocation registers.

Performance Trade-off: It is a high-overhead operation as it involves copying large amounts of data in physical memory.

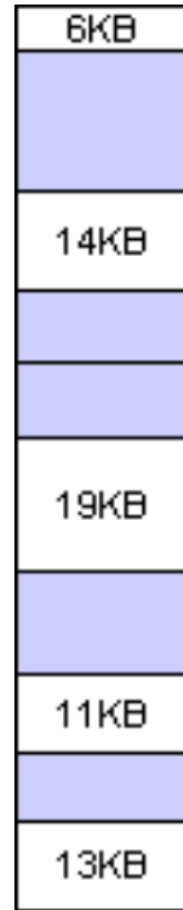
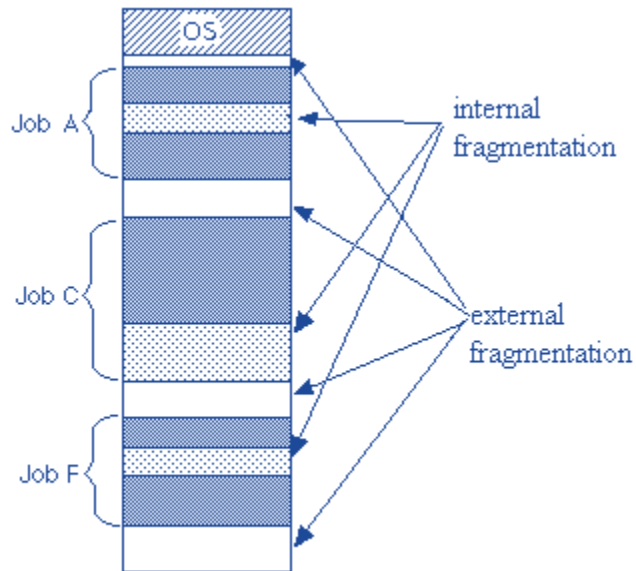
Note:

Compaction cannot be used with static binding (compile/load time) because physical addresses are changed during the move.

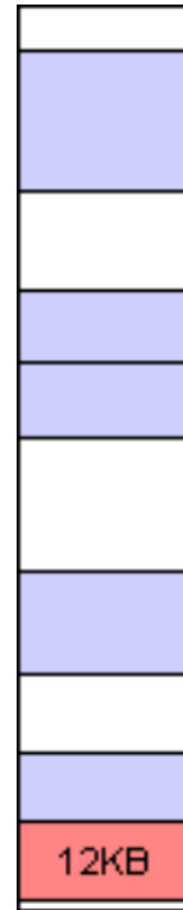


Placement Algorithms

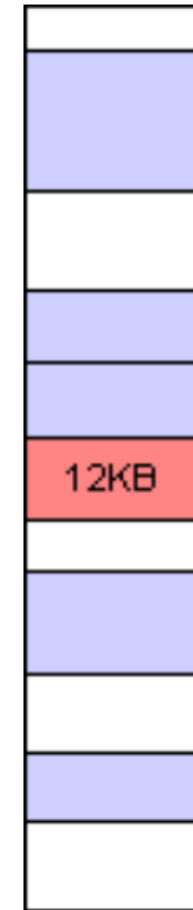
1. First Fit
2. Best Fit
3. Worst Fit



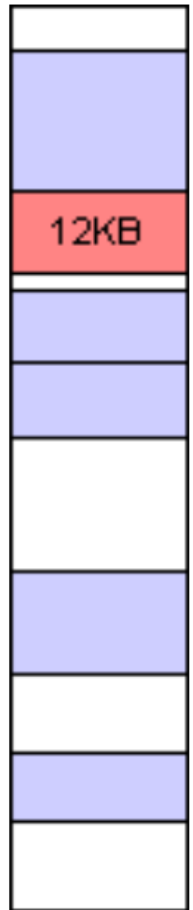
Primary
Memory



Best fit



Worst fit



First fit

Non- Contiguous



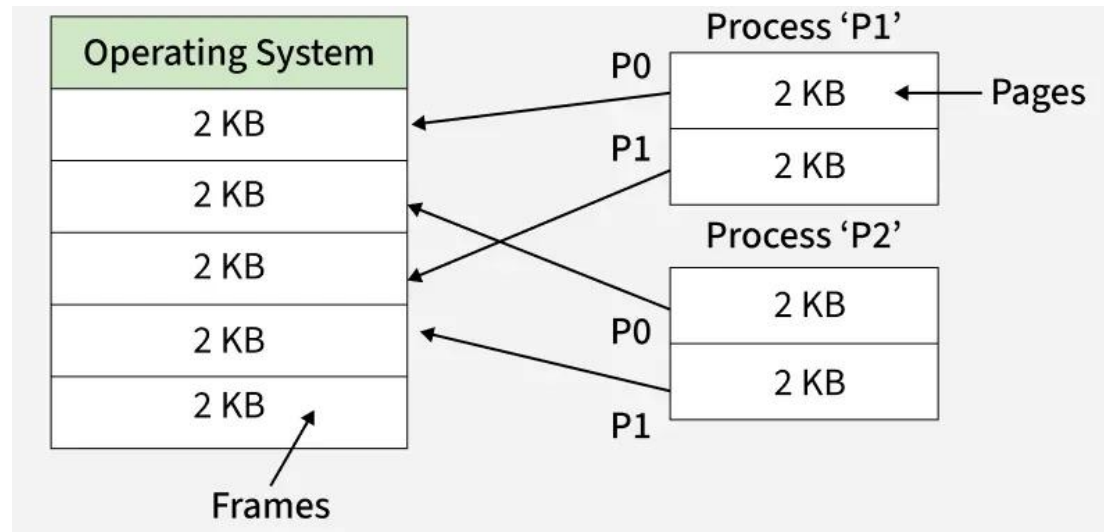
Fragmentation Issue

Contiguous allocation leads to **external fragmentation**, where total free memory exists but isn't available in one contiguous block.



The Solution

Allowing a process's physical address space to be non-contiguous. We split the process into smaller pieces and map them to available holes.

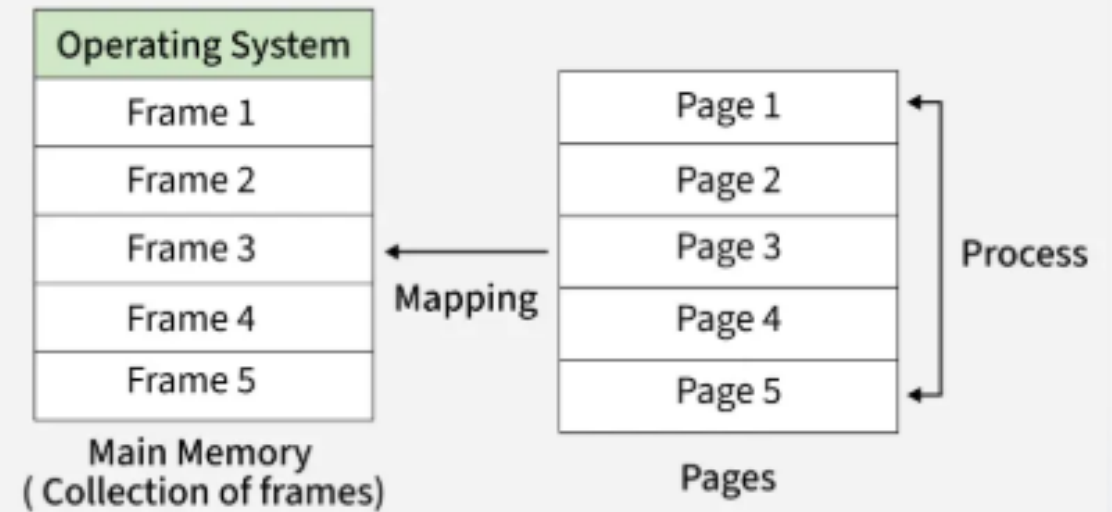


Paging

Pages: Logical memory is broken into fixed-size blocks of the same size.

Frames: Physical memory is divided into fixed-size blocks called frames.

The OS maintains a **Page Table** to translate logical pages to physical frames.



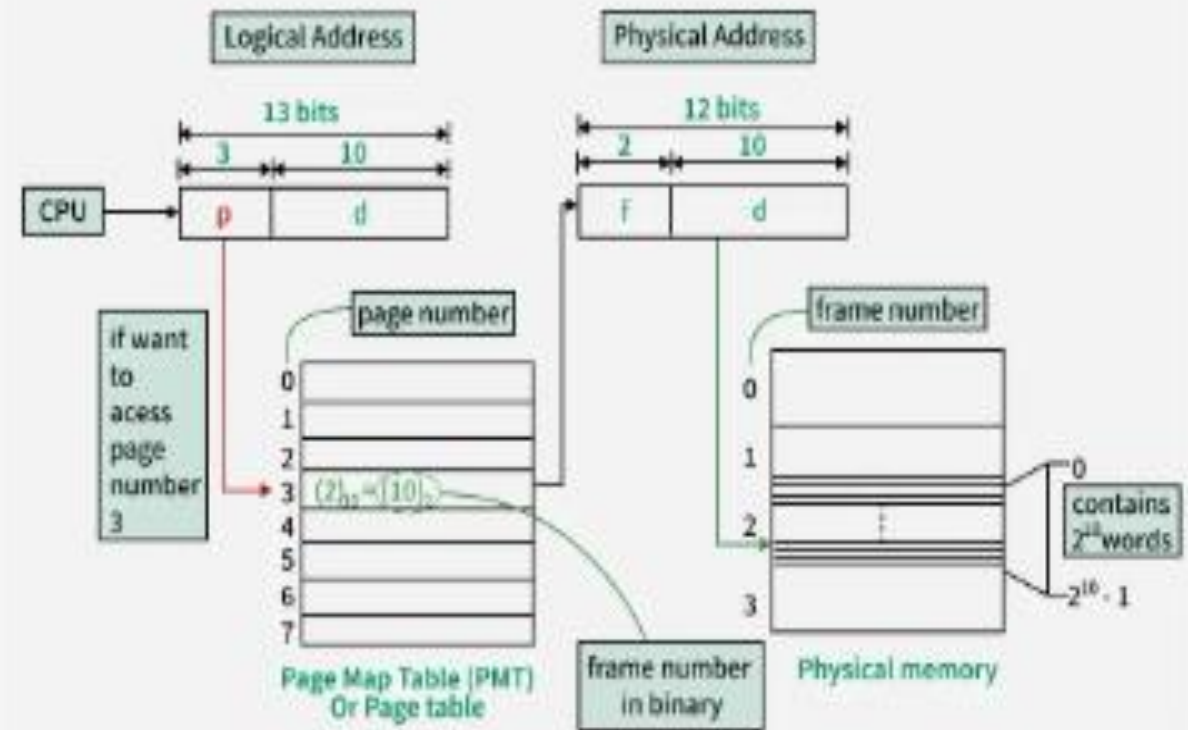
Address Translation Scheme

CPU generates a logical address divided into two parts:

- **Page Number (p):** Index into the page table.
- **Page Offset (d):** Specific location within the page/frame.

$$\text{Physical Address} = (f \times S) + d$$

Where f = Frame #, S = Page Size, d = Offset



Why Use Paging?

1. Zero External Fragmentation: Any free frame can be used for any page, regardless of its location in memory.
2. Easy Allocation: OS just needs to keep a list of free frames; no complex searching like Best-Fit or Worst-Fit.
3. Internal Fragmentation: Only occurs in the last page of a process if the process size isn't a multiple of page size.
4. Separation of Views: User sees memory as contiguous; hardware sees it as scattered frames.

Translation Lookaside Buffer (TLB)



The Problem

Standard paging requires 2 memory accesses (one for table, one for data), slowing down the CPU significantly.



The Solution (TLB)

Translation Lookaside Buffer: A fast, associative cache that stores recently used page-to-frame mappings.



Hit Ratio

If the page number is in TLB (Hit), translation is near-instant. Most programs exhibit high locality of reference.

Question

Consider a computer system with the following memory specifications:

- Logical Address Space: 32-bit virtual addresses
- Physical Memory (RAM): 16 MB (Megabytes)
- Page Size: 8 KB (Kilobytes)
- Process 'X' Size: 50 KB

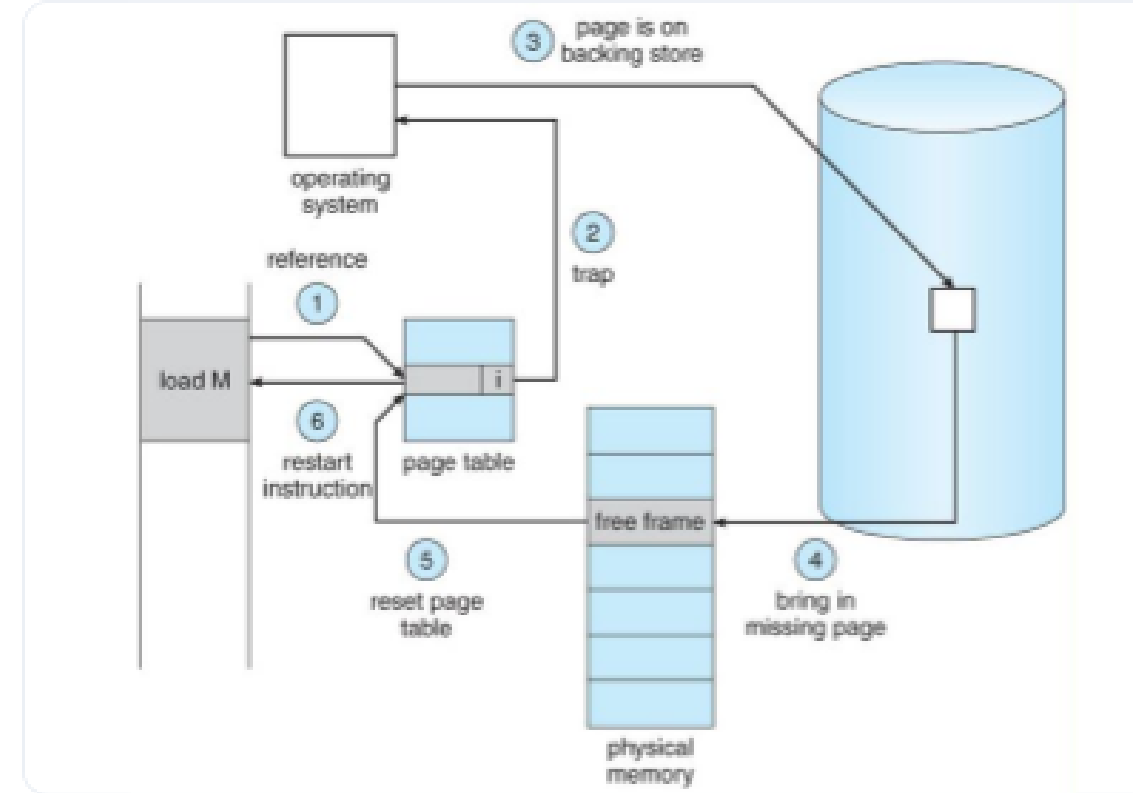
Calculate the following:

1. How many bits are required for the Page Offset (d)?
2. How many bits are required for the Page Number (p)?
3. What is the total number of Virtual Pages possible in this system?
4. How many bits are required to represent a Physical Address?
5. What is the total number of Physical Frames available in the RAM?
6. How many pages will Process 'X' require, and how much internal fragmentation (wasted space) will it experience?

What is a Page Fault?

A **Page Fault** occurs when the CPU attempts to access a page marked as 'invalid' in the page table (not in RAM).

- **Hardware Trap:** The MMU sends an interrupt to the OS.
- **Storage Fetch:** The missing page must be loaded from secondary storage (Disk/SSD).
- **Process Delay:** The process enters a 'waiting' state during I/O.



Effective Access Time (EAT)

EAT measures the actual speed of memory access including the overhead of page faults.

$$\text{EAT} = (1 - p) \times \text{ma} + p \times \text{PFST}$$

p = fault rate, ma = memory access time, PFST = fault service time.

Example: If $\text{ma} = 100\text{ns}$ and $\text{PFST} = 8\text{ms}$, even a 0.1% fault rate ($p=0.001$) increases access time to **8,100ns** (81x slower)!

Thank you

Any Questions?